Algoritmi e Strutture Dati – Prova di Laboratorio

16/04/2026

Istruzioni

Risolvere il seguente esercizio implementando un programma in un singolo file .cpp, completo di funzione *main*. Si presti particolare attenzione alla formattazione dell'input e dell'output, e alla complessità target indicata per ciascuna funzionalità. Nel caso la complessità target non sia specificata, si richiede che sia la migliore possibile. La lettura dell'input e la scrittura dell'output **DEVONO** essere effettuate tramite gli stream **cin** e **cout** rispettivamente. La correzione avverrà prima in maniera automatica inviando il file .cpp al server indicato in aula. Quest'ultimo esegue dei test confrontando l'output prodotto dalla vostra soluzione con l'output atteso. In caso la verifica abbia esito positivo sarà possibile consegnare il compito, il quale verrà valuato dai docenti in termini di complessità. Si ricorda che è possibile testare la correttezza del vostro programma in locale su un sottoinsieme dei input/output utilizzati nella seguente maniera. I file di input e output per i test sono nominati secondo lo schema: `input0.txt output0.txt input1.txt output1.txt ...` Per effettuare le vostre prove potete utilizzare il comando del terminale per la redirezione dell'input. Ad esempio

```
./compilato < input0.txt
```

effettua il test del vostro codice sui dati contenuti nel primo file di input, assumendo che `compilato` contenga la compilazione della vostra soluzione e che si trovi nella vostra home directory. Dovete aspettarvi che l'output coincida con quello contenuto nel file `output0.txt`. Per effettuare un controllo automatico sul primo file input `input0.txt` potete eseguire la sequenza di comandi

```
./compilato < input0.txt | diff - output0.txt
```

Questa esegue la vostra soluzione e controlla le differenze fra l'output prodotto e quello corretto.

Esercizio

In uno scenario di crisi energetica, si devono pianificare le rotte di aerei che partono da un Hub principale e si muovono attraverso una rete di aeroporti. Il sistema memorizza le rotte all'interno di un Albero Binario di Ricerca (ABR). Il numero di aeroporti da memorizzare è N.

Ogni nodo x dell'albero rappresenta un aeroporto ed è caratterizzato da: i) una stringa codice $IATA$ che rappresenta il codice identificativo dell'aeroporto; ii) un intero $Dep(x)$ che rappresenta cherosene disponibile nel deposito locale per il rifornimento di un aereo; iii) un intero $KM(x)$ che indica distanza chilometrica che separa il nodo x dal suo nodo padre; iv) ed un intero $C_{km}(x)$ che indica il consumo di carburante al chilometro per la tratta dal padre a x.

L'inserimento avviene utilizzando il campo $IATA$ come etichetta. Per la radice, si consideri sempre $KM(radice) = 0$. L'aereo parte con un quantitativo di carburante iniziale CI_{Iniz}.

Si definisca il **Carburante Residuo** $CR(x)$ come il quantitativo di cherosene che l'aereo possiede al nodo x dopo il rifornimento. Esso è calcolato ricorsivamente come:

$$CR(x) = CR(padre) - (C_{km}(x) \cdot KM(x)) + Dep(x)$$

dove per la radice $CR(radice) = CI_{Iniz} + Dep(radice)$.

Un nodo x si dice **Critico** se il suo carburante residuo $CR(x) \leq 0$ oppure se almeno uno dei suoi discendenti è a sua volta Critico.

Scrivere un programma che:

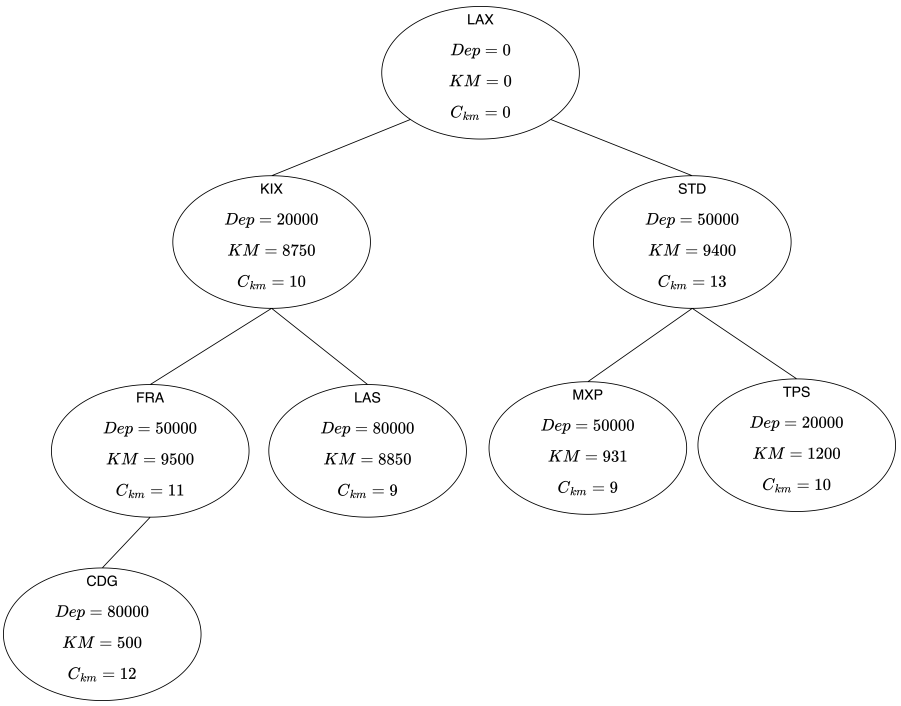
- legga da tastiera N quadruple $\{IATA, Dep, KM, C_{km}\}$ e le inserisca all'interno dell'ABR. I nodi devono essere inseriti nello stesso ordine con cui vengono letti. Per l'inserimento, le etichette dei nodi vanno confrontate utilizzando l'ordine lessicografico (le etichette $\leq$ vanno inserite a sinistra);
- Identifichi tutti i nodi **Non Critici**.
- Stampi il codice $IATA$ di tali nodi in ordine **decrescente**. Se non esistono nodi non critici, si stampi -1.

L'**input** è formattato nel seguente modo: la prima riga contiene gli interi N e CI_{Iniz}. Seguono N righe contenenti una quadrupla $stringa, intero, intero, intero$ ciascuna. L'**output** contiene gli elementi della soluzione, uno per riga.

Esempio

Input

```
8 100000
LAX 0 0 0
KIX 20000 8750 10
STD 50000 9400 13
TPS 20000 1200 10
LAS 80000 8850 9
MXP 50000 931 9
FRA 50000 9500 11
CDG 80000 500 12
```

Output

```
TPS
STD
MXP
LAS
CDG
```